

SOFTWARE ENGINEERING

SUBJECT CODE: MCAXX11509

UNIT 5 : SOFTWARE REQUIREMENT SPECIFICATION [SRS]



UNIT OUTLINE

- Introduction of software requirement Specification
- Need of SRS/ Role of SRS
- Component of SRS
- Data Flow Diagram
- Entity Relationship Diagram
- Decision table
- SRS Document



WHAT IS SOFTWARE REQUIREMENT SPECIFICATION (SRS)?

- **SRS (Software Requirement Specification)** is a document in which **all the user requirements are mentioned in a structural manner.**
- SRS is a kind of **agreement between the customer and the company**, which contains complete information about the requirement desired by the customer and the product made by the company.
- In other words, SRS is a document which **describes what will be the features of the software and what will be its behavior.** That is, what kind of performance he will perform.



NEED OF SRS /ROLE OF SRS

1. Clarity and Understanding:

- The SRS provides a common communication channel between stakeholders, so that everyone agrees as to what the project aims at.
- It reduces the risks of misunderstandings among team members in clearly defining requirements.

2. Basis for Project Planning:

- The SRS is the required information for project managers to define realistic timelines, assign appropriate resources and set milestones.
- It provides a basis for planning projects, which allows teams to accurately estimate costs and schedules.



3. Guidance for Development Teams:

- The SRS is **used to guide developers** in their understanding of the functional specifications and how the software will be designed.
- The **document offers an overview of the system architecture, user interfaces and data structures** which enable developers to build a system that meets with client expectations.

4. Change Management:

- **During projects, changes are unavoidable.**
 - **The SRS thus provides a baseline from which any proposed changes can be judged.**
 - This guarantees that changes are thoroughly recorded, approved and properly implemented--without scope creep.
- 

5. Client and Stakeholder Satisfaction:

- The SRS is a contract between the development team and their client or stakeholders.
- This boosts client satisfaction and trust if the final product meets what was documented

6. Risk Mitigation:

- The earlier in the software development life cycle you can spot potential risks.
- The SRS helps in risk assessment by specifying dependencies, constraints and possible risks.
- This makes the development process smoother for project managers who can then come up with strategies to prevent risk.

7. Enhanced Collaboration:

- Software development requires collaboration.
- The SRS has team spirit Working together with cross-functional teams, it forms a single vision and shared understanding of the project.
- As a result, the development process becomes more integrated and efficient.



COMPONENT OF SRS

- The important parts of the Software Requirements Specification (SRS) document are:
 1. Functional requirements of the system
 2. Non-functional requirements of the system, and
 3. Goals of implementation



1. Functional Requirements

- In a **Software Requirements Specification (SRS)** document, **Functional Requirements** define the specific **functionalities and capabilities that a system must have to meet user needs**. These requirements outline **what** the system should do, rather than **how** it should do it.

➤ Key Aspects of Functional Requirements

- **Description of Features** – **Specifies the core functionalities** of the system.
- **User Interaction** – Defines **how users interact with the system** (e.g., login, data entry, reports).
- **System Behavior** – Details system **responses to inputs, processing rules, and expected outputs**.
- **Dependencies & Constraints** – Specifies any **dependencies on hardware, software, or third-party services**.



2. Non-functional Requirements

- In a **Software Requirements Specification (SRS)** document, **Non-Functional Requirements (NFRs)** define the quality attributes, constraints, and performance criteria of a system.
- **Key Aspects of Non-Functional Requirements**
- **Performance** – Defines **system response time, speed, and scalability.**
- **Security** – Specifies **authentication, authorization,** and data protection measures.
- **Usability** – Ensures the **system is user-friendly** and easy to navigate.
- **Maintainability & Scalability** – Ensures the **system can be easily updated and expanded.**
- **Compatibility** – Defines system **compatibility with different hardware, software, or networks.**



3. Goals of Implementation

- The goals of **implementation** part documents some **general suggestions relating to development.**
- The goals of the implementation section would possibly document problems like ...
 - ❖ **revisions to the system functionalities that will be needed within the future,**
 - ❖ **new devices to be supported within the future,**
 - ❖ **reusability problems, etc.**



Data Flow Diagrams (DFDs)

- A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system.
- A neat and clear DFD can depict the right amount of the system requirement graphically.
- It can be manual, automated, or a combination of both.
- It shows how data enters and leaves the system, what changes the information, and where data is stored.
- The objective of a DFD is to show the scope and boundaries of a system as a whole.
- The DFD is also called as a data flow graph or bubble chart.



SYMBOLS IN DFDs:

1. **Circle (Process):** Represents an activity or function (e.g., "Calculate Grade").
2. **Arrow (Data Flow):** Represents data movement between entities, processes, or stores.
3. **Rectangle (External Entity):** Represents entities outside the system (e.g., "Customer").
4. **Open Rectangle (Data Store):** Represents where data is stored (e.g., "Database").

	dataflow	Arrows showing direction of flow
	process	circles
	file	horizontal pair of lines
	data-source, sink	rectangular box

➤ **Levels in Data Flow Diagram (DFD)**

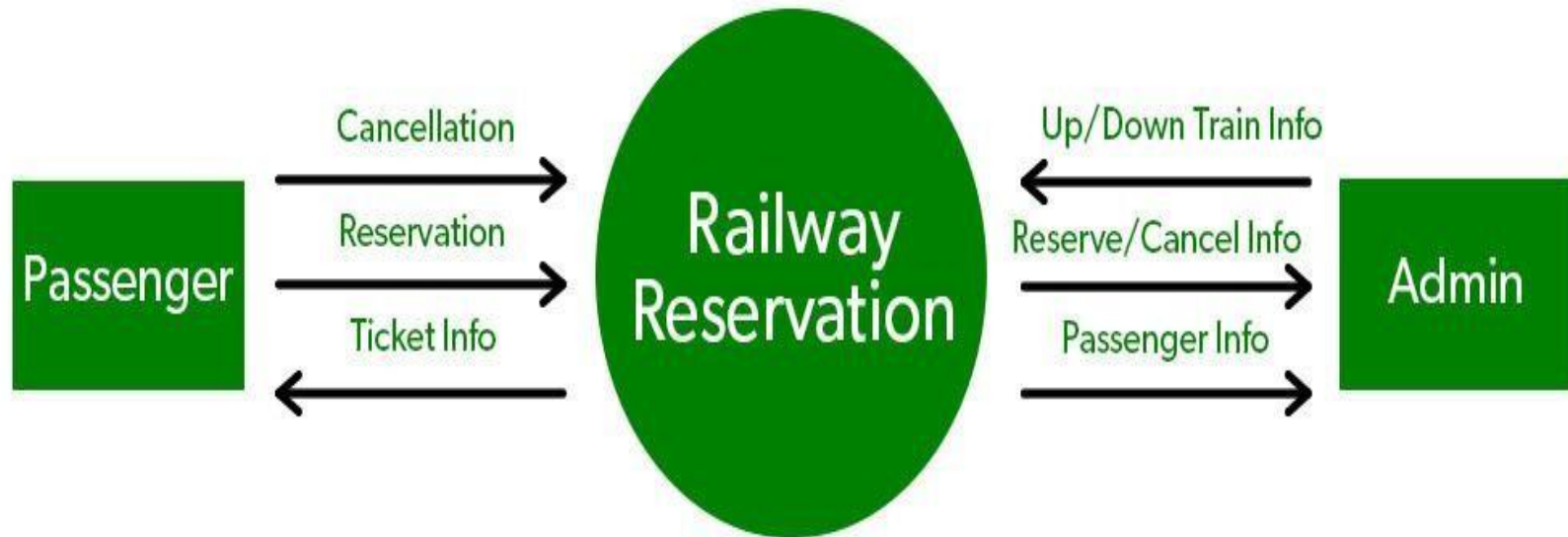
- DFDs can be divided into different levels, which provide varying degrees of detail about the system. The following are the four levels of DFDs:
- **Level 0 DFD**
- **Level 1 DFD**
- **Level 2 DFD**
- **Level 3 DFD**



➤ Level 0 Data Flow Diagram (DFD)

- Level 0 is the highest-level Data Flow Diagram (DFD), **which provides an overview of the entire system.**
- It shows the **major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes.**
- It is also known as a **context diagram.**



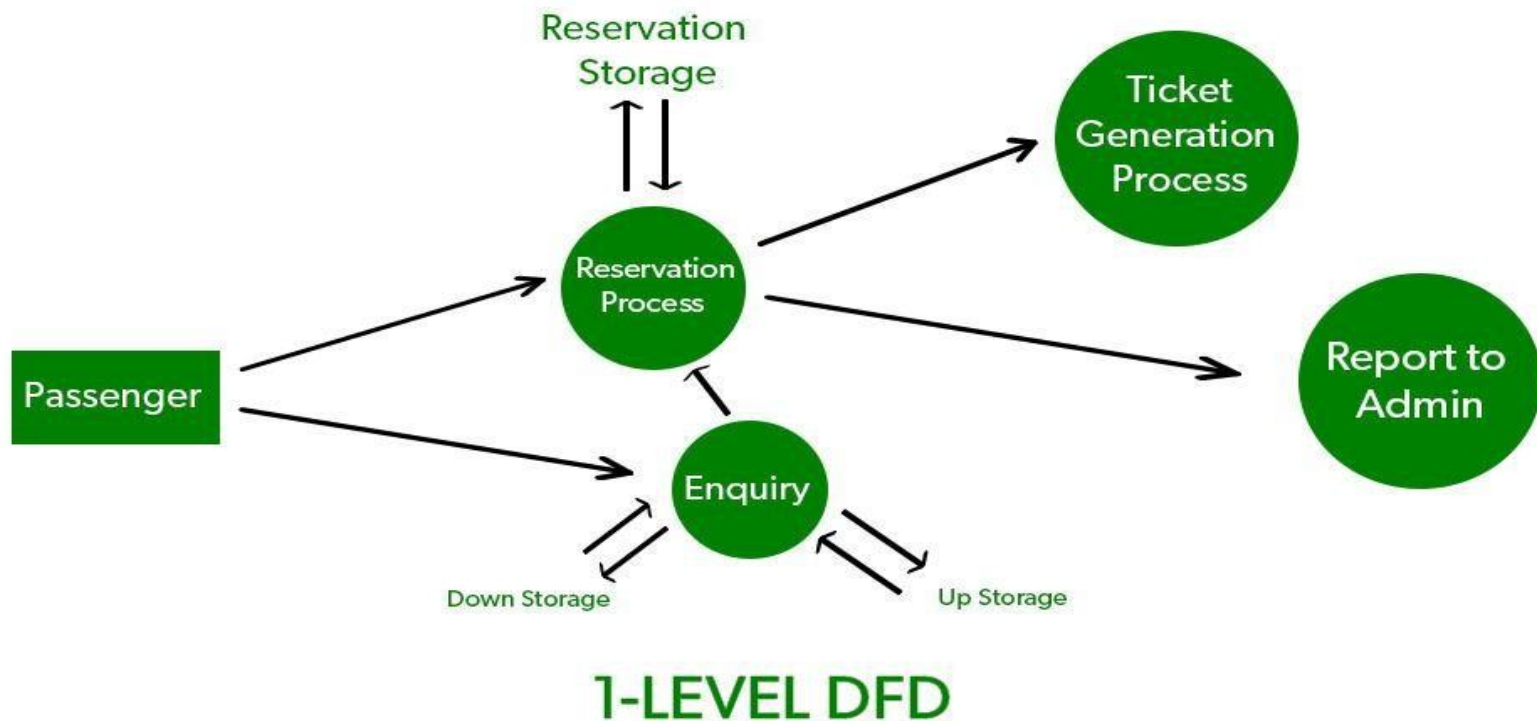


0-LEVEL DFD



➤ 1-Level Data Flow Diagram (DFD)

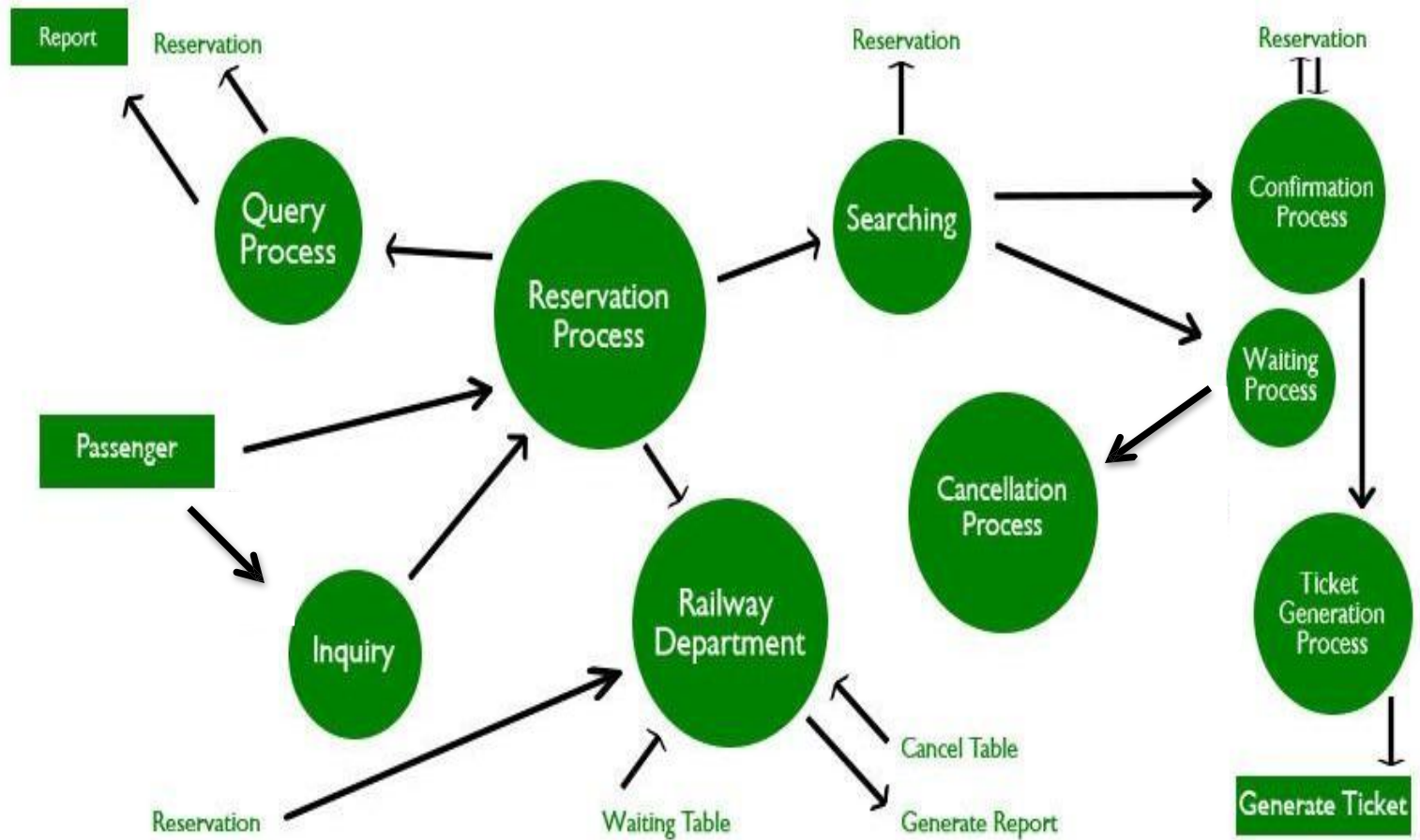
- 1-Level provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes.
- Each sub-process is depicted as a separate process on the level 1 Data Flow Diagram (DFD).



➤ 2-Level Data Flow Diagram (DFD)

- 2-Level provides an even more detailed view of the system by breaking down the sub-processes identified in the level 1 Data Flow Diagram (DFD) into further sub-processes.
- Each sub-process is depicted as a separate process on the level 2 DFD.
- 2-Level Data Flow Diagram (DFD) goes one step deeper into parts of 1-level DFD.
- It can be used to plan or record the specific/necessary detail about the system's functioning.





2-LEVEL DFD

➤ 3-Level Data Flow Diagram (DFD)

- 3-Level is the **most detailed level of Data Flow Diagram (DFDs)**, which provides a detailed view of the processes, data flows, and data stores in the system.
- This level is **typically used for complex systems**, where a high level of detail is required to understand the system.
- Each process on the **level 3 DFD** is depicted with a detailed description of its input, processing, and output.



➤ Advantages of using Data Flow Diagrams (DFD)

1. **Easy to understand:** DFDs are graphical representations that **are easy to understand and communicate, making them useful for non-technical stakeholders and team members.**
2. **Improves system analysis:** DFDs are useful for analyzing a system's processes and data flow, which can help **identify inefficiencies, redundancies, and other problems** that may exist in the system.
3. **Supports system design:** DFDs can be used to design a system's architecture and structure, which can help ensure that **the system is designed to meet the requirements of the stakeholders.**
4. **Facilitates documentation:** DFDs provide a visual representation of a system, **making it easier to document and maintain the system over time.**



➤ **Disadvantages of using Data Flow Diagram (DFD)**

1. **Can be time-consuming:** Creating DFDs can be a time-consuming process, especially for complex systems.
2. **Can be difficult to keep up-to-date:** DFDs may become out-of-date over time as the system evolves and changes.
3. **Requires technical expertise:** While DFDs are easy to understand, creating them requires a certain level of technical expertise and familiarity with the system being analyzed.

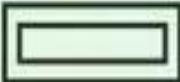


ENTITY RELATIONSHIP DIAGRAM

- ER diagrams represent the **E-R model in a database**, making them easy to convert into relations (tables).
- ER diagrams provide the **purpose of real-world modeling** of objects which makes them intently useful.
- ER diagrams require no technical knowledge of the underlying DBMS used.
- It gives a **standard solution for visualizing the data logically**.

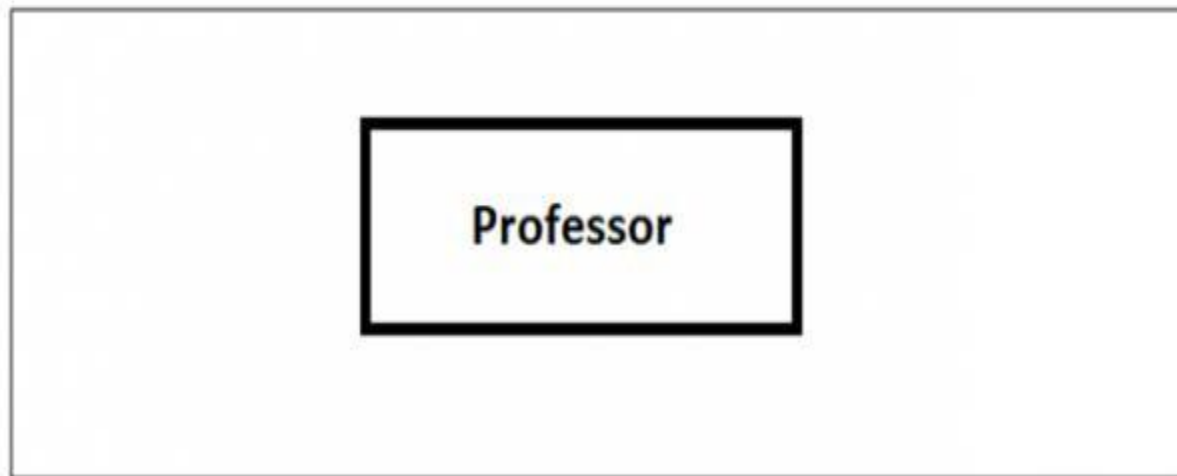


SYMBOLS USED IN ER MODEL

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

TYPES OF ENTITY RELATIONSHIP IN DBMS

- Entity in DBMS can be a real-world object with an existence, For example, in a **Company** database, the entities can be **Employees, Department, Project**, etc. In a **College** database, the entities are **Profession, Students, Result, Activities**, etc.
- An entity is represented as a single rectangle, shown below



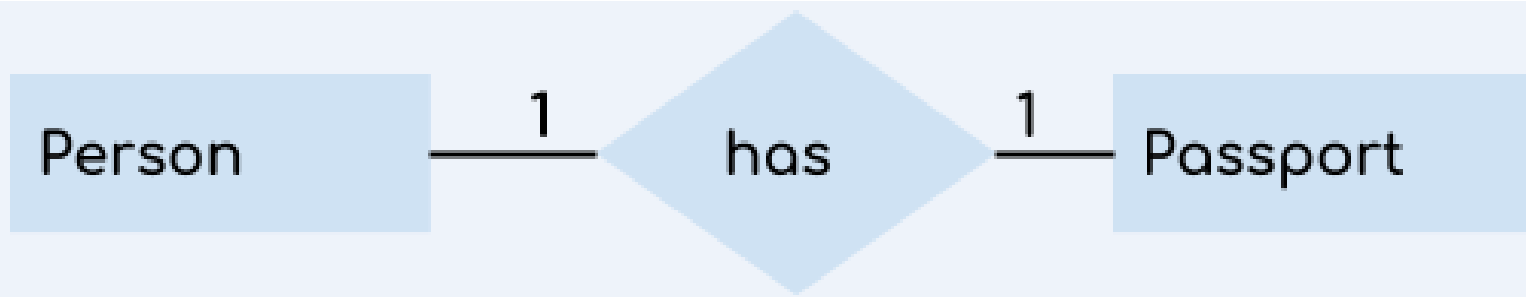
○ The following are the entity relationships –

1. **One-to-One Relationship**
2. **One-to-Many Relationship**
3. **Many-to-one Relationship**
4. **Many-to-Many Relationship**



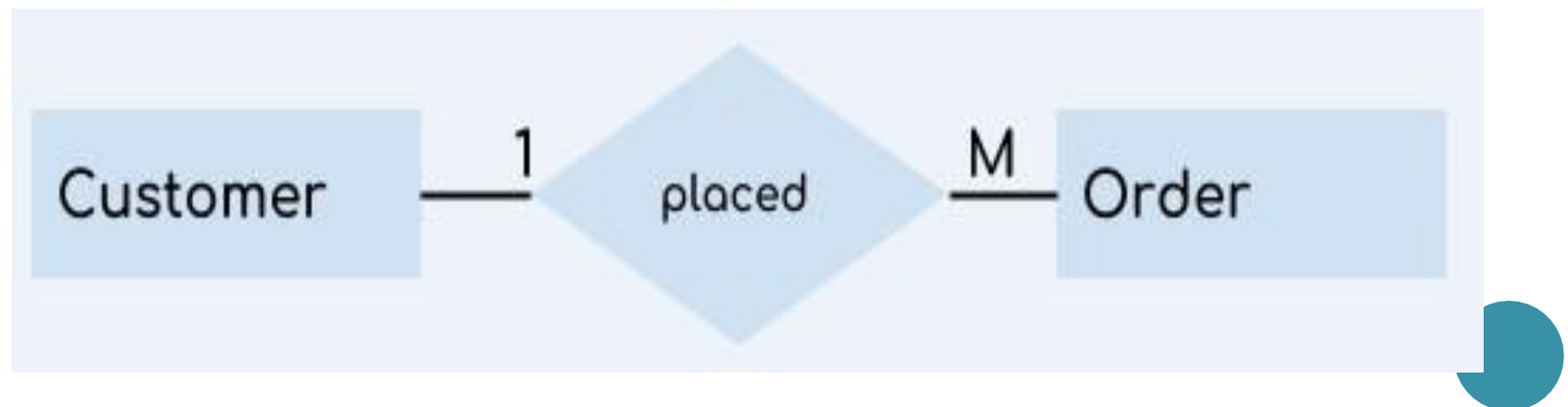
1. One-to-One Relationship:

- When a single instance of an entity is associated with a single instance of another entity then **it is called one to one relationship.**
- For example, a person has only one passport and a passport is given to one person.



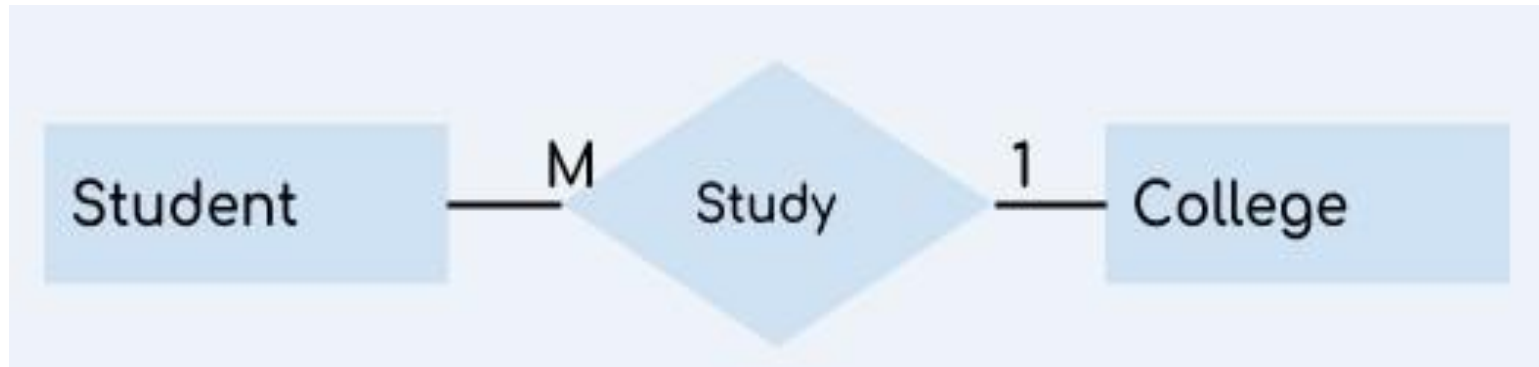
2. One to Many Relationship:

- When a single instance of an entity is associated with more than one instances of another entity then it **is called one to many relationship**.
- For example – a customer can place many orders but a order cannot be placed by many customers.



3. Many to One Relationship

- When more than one instances of an entity is associated with a single instance of another entity then it is **called many to one relationship**.
- For example – many students can study in a single college but a student cannot study in many colleges at the same time.

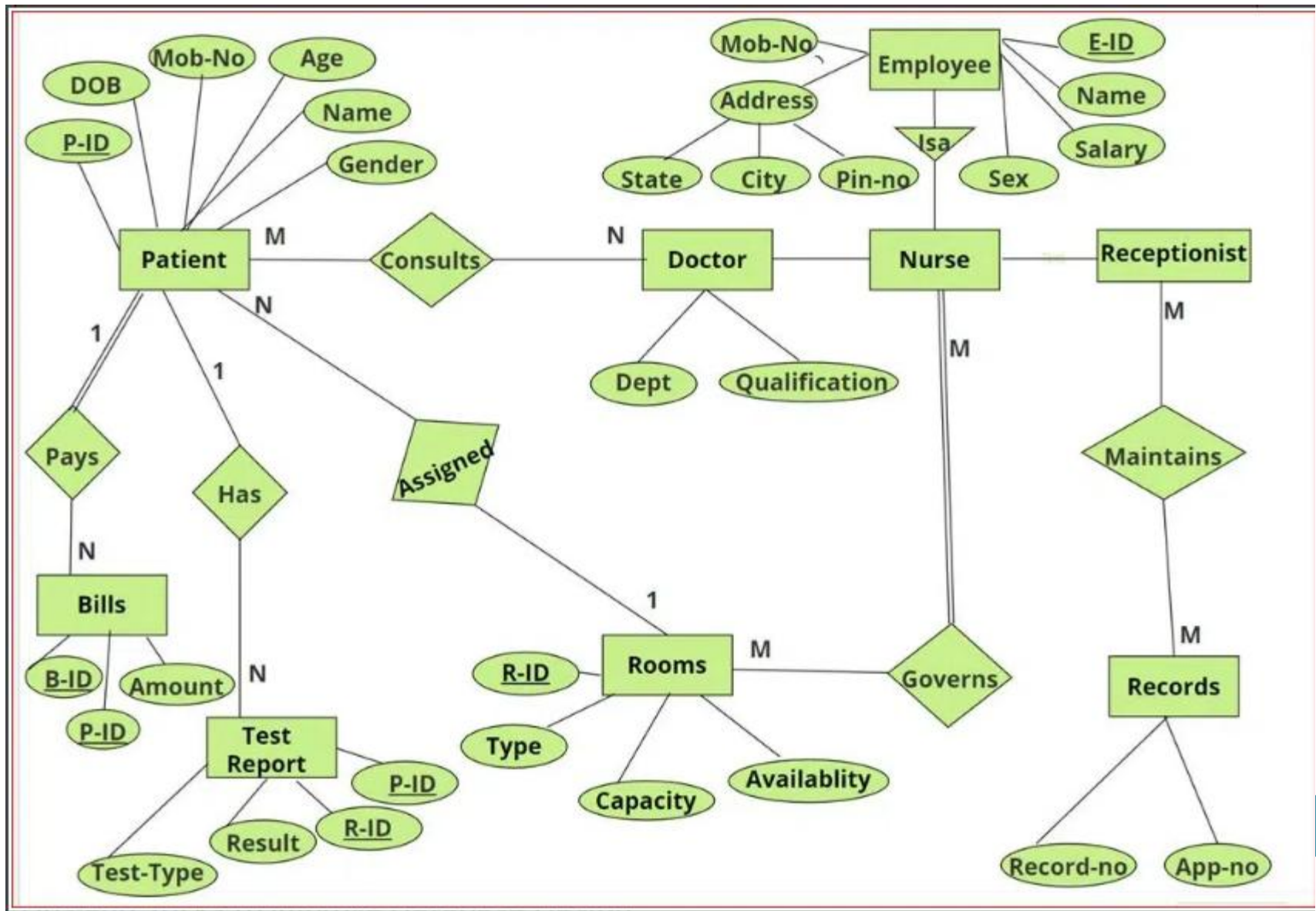


4. Many to Many Relationship

- When more than one instances of an entity is associated with more than one instances of another entity then **it is called many to many relationship.**
- For example, a can be assigned to many projects and a project can be assigned to many students.



HOSPITAL MANAGEMENT ER DIAGRAM



DECISION TABLE

- A Decision Table is a table that shows the relationship between inputs and rules, cases, and test conditions.
- It's a very useful tool for both complicated software testing and requirements management.
- The decision table allows testers to examine all conceivable combinations of requirements for testing and to immediately discover any circumstances that were overlooked.
- True(T) and False(F) values are used to signify the criteria.



WHAT IS DECISION TABLE TESTING?

- Decision table testing is a type of software testing that examines how a system responds to various input combinations.
- That's why it's also known as a Cause-Effect table because it captures both causes and effects for improved test coverage.



- **Example 1 – How to Create a Login Screen Decision Base Table**
- login screen with a decision table
- A login screen with E-mail and Password Input boxes.
- **The condition is simple** – The user will be routed to the homepage if they give the right username and password. An error warning will appear if any of the inputs are incorrect.



Conditions	Test case 1	Test case 2	Test case 3	Test case 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

❖ Legend:

T - Make sure your login and password are correct.

F - Incorrect login or password

E - An error message appears.

H - The home screen appears.

❖ Interpretation

Case 1 – Both the username and password were incorrect. An error message is displayed to the user.

Case 2 – The username and password were both right, however, the password was incorrect. An error message is displayed to the user.

Case 3 – Although the username was incorrect, the password was accurate. An error message is displayed to the user.

Case 4 – The user's username and password were both accurate, and the user went to the homepage.

SRS Document

- The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**).
- This report lays a foundation for software engineering activities and is constructing when entire requirements are elicited and analyzed.
- SRS** is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements.
- Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.



1. Introduction

1. Purpose
2. Intended Audience
3. Intended Use
4. Product Scope
5. Definitions and Acronyms

2. Overall Description

1. User Needs
2. Assumptions and Dependencies

3. System Features and Requirements

1. Functional Requirements
2. External Interface Requirements
3. System Features
4. Nonfunctional Requirements



SRS Document For Library Management System

1. Introduction

1. Purpose

- The purpose of this document is to **describe the software requirements for the Library Management System.**
- This system will **support the librarians, students, and administrative** staff in managing the catalog of books, issuing and returning of books, maintaining member records, and generating reports.

1.2 Intended Audience

- ❖ **Librarians** who will manage the system operations.
 - ❖ **Students/Users** who will search and borrow books.
 - ❖ **System Developers** and **Testers** who will use this document to understand requirements.
 - ❖ **Project Managers** and **Stakeholders** for planning and monitoring the project.
- 

1.3 Intended Use

This system will be used to

- Add, update, or delete book records.
- Issue and return books.
- Manage user accounts.
- Track book availability and due dates.
- Generate reports for administration and audits.

1.4 Product Scope

- The Library Management System automates the process of a traditional library with **improved efficiency and accuracy**.
- It **manages book inventory, member subscriptions, lending/return operations, overdue notifications, and report generation**.



1.5 Definitions and Acronyms

- **LMS**: Library Management System
- **ISBN**: International Standard Book Number
- **GUI**: Graphical User Interface
- **DBMS**: Database Management System
- **CRUD**: Create, Read, Update, Delete



2. Overall Description

1. User Needs

- Users need a simple interface to search and borrow books.
- Librarians need to manage the catalog and keep track of borrowed/returned books.
- Administrators need detailed reports and analytics.
- Users need reminders for due dates and late return penalties.

2.2 Assumptions and Dependencies

- The system will be accessible through web browsers.
- The application depends on a secure and operational DBMS.
- Users must have valid login credentials to access the system.



3. System Features and Requirements

1. Functional Requirements

- **FR1:** The system shall allow librarians to add, update, and remove book records.
- **FR2:** The system shall allow users to search for books by title, author, or ISBN.
- **FR3:** The system shall manage issuing and returning of books.
- **FR4:** The system shall send overdue alerts to users.
- **FR5:** The system shall generate daily, weekly, and monthly reports.



3.2 External Interface Requirements

○ User Interface:

- Responsive web interface for desktop and mobile.
- Login and registration screens.
- Dashboards for users and librarians.

○ Hardware Interface:

- Compatible with barcode scanners for book issuing and return.

○ Software Interface:

- Connects with relational DBMS like MySQL.
- Integration with email or SMS API for notifications.



3.3 System Features

- **Book Management:** Add, edit, delete, and search books.
- **User Management:** Register, update, and manage user accounts.
- **Transaction Management:** Issue, return, and track books.
- **Fine Management:** Calculate and display overdue fines.
- **Reporting:** Generate usage and transaction reports.

3.4 Nonfunctional Requirements

- **Performance:** The system should **handle up to 1000 concurrent users.**
 - **Reliability:** **99.9% uptime expected.**
 - **Security:** **Password encryption, role-based access control, and secure communication (HTTPS).**
 - **Scalability:** System should **support expansion to multiple library branches.**
 - **Usability:** The system should be **user-friendly and accessible to non-technical users.**
- 

THANK YOU

